Engineering Principles

Engineering Principles

At GitLab, Company Culture is very important to us. The main ingredient of the company culture are GitLab Values.

GitLab Values have guided us throughout the evolution of the company. Those values have been crucial in maintaining a positive and productive culture, helping us make decisions to make the company and the product better.

Our Engineering Principles are built on top of GitLab Values, and provide additional explanation of these in the context of the software engineering practice.

1. Iteration
2. Efficiency
3. Simplicity
4. Reliability
5. Quality
6. Velocity
7. Predictability

Iteration

We always push ourselves to be iterative and make the minimal valuable change that is on the direct path to achiving our goals.

For complex initiatives we are using the Architecture Design Workflow to define our goals and to describe our iteration plan for a given effort. How we are going to iterate on something may depend on our mid-to-long-term vision. Our goal is to make the iteration as efficient and as pragmatic as possible.

We also sometimes use an established “scooter heuristic”: release a smallest product that our customers can use, and then iterate on it.

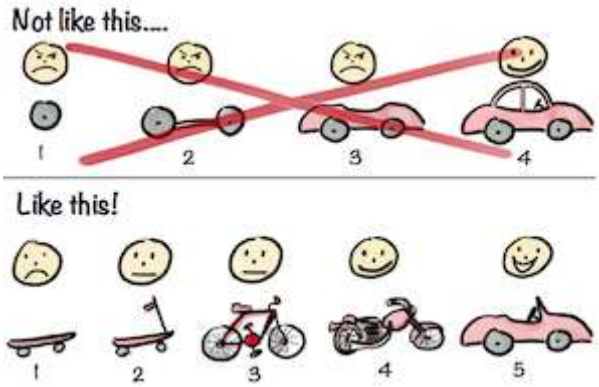


Image Credit: Henrik Kniberg from Crisp

Releasing minimal changes, as quickly as possible, is sometimes a good way to think about iteration, especially when intermediate steps provide value to our users and allow us to collect meaningful feedback. This has often been our default iteration approach.

This “scooter” iteration pattern, however, is not always the most efficient way to make progress on something. Sometimes we do need to have a tailored plan for efficient iteration, because if we want to build something more complex, our users may not be happy with a scooter or a paper airplane, and we will either not get any meaningful feedback or it will not be significant enough to be useful. In such cases, how we iterate should be a diligently planned strategy, designed to improve efficiency and to allow us to achieve our goals within a reasonable amount of time.

At GitLab we can use our Architecture Design Workflow to build a more pragmatic iteration plan aligned with mid-to-long term vision. This usually covers building a Minimal Viable Product too. In this case, however, how the MVP is going to look like depends on a thoughtful plan, described in writing after collecting feedback throughout the organization.

Below you can find a few patterns that can precede “building the smallest thing in the next milestone” pattern, aimed to increase iteration efficiency:

1. Identify fundamental design problems, describe these in a design doc.
2. Identify most significant risks associated with those, find ways to de-risk.
3. Develop a plan to iterate on the fundamental design aspects individually.
4. Deliver a first end-to-end iteration as a prototype or a Proof-of-Concept.

All these things attempt to improve our level of knowledge and reduce implementation risk. All of these steps are also iterations by themselves that can help us find hidden efficiencies to build better MVPs.

To read more about iteration, see our values page and engineering workflow.

Efficiency

Efficiency is at the core our engineering discipline at GitLab. This is also one of the most important company values.

How we work in Engineering can have a significant compounding effect over the mid-to-long term. If we constantly ask ourselves, "How can we implement this thing to become more efficient in mid-to-long term?" while still thinking about pragmatic short-term goals, we can hope for seeing a lot value being built over time, similarly to how compound interest builds wealth. Overfocusing on short-term goals can lead to us missing out on this opportunity.

A short, and incomplete list of things that can help us become more efficient:

1. Build useful and well designed abstractions.
2. Choose simple solution over easy solutions.
3. Refactor code for simplicity and ease of understanding.
4. Contribute to an open source project instead of patching locally.
5. Build automation to reduce repetitive work or simplify a process.
6. Build new communication channels that enhance knowledge sharing.
7. Strive for achieving engineering excellence over engineering perfection.

Simplicity

Simplicity is a prerequisite for reliability. It is also at the core our boring solutions value. We strive for choosing simple solution over easy solution, even when it sometimes means that we will need to put more work into building a simple solution.

Reliability

Availability/Reliability, **Quality**, **Security**, and **Performance** are the pillars for building reliable software. Reliability is our contract with our customers that say you can count on us to deliver an available and dependable product. Everyone in the organization has a role to play.

Engineers, Product Managers, and Designers have the most direct influence over the reliability of the code through either planning, implementation, monitoring (e.g. Kibana, Sentry, Grafana and other GitLab.com monitoring tools), or prioritization of the work. Product and Engineering management monitors (e.g. Error Budgets) and measures the reliability of features and makes recommendations if necessary. Our focus on learning and development will also ensure that teams have the tools and training required to build reliable software. The Infrastructure

Platforms, Application Security, Database and Developer Experience teams are the Subject Matter Experts supporting product development teams.

Velocity

Our velocity should be incremental in nature. It's derived from our Iteration and Efficiency principles.

We recognize the importance of keeping our velocity high:

- The rate at which GitLab delivers new value to users is a competitive advantage for the company.
- As an open source project, people are welcome to fork us. However, in order to ensure that the community contributing to GitLab remains intact it is important to move fast so that any fork is quickly out-of-date.
- Once you slow down, it is incredibly painful to speed back up again.
- Companies tend to slow down as they grow. It takes deliberate effort to prevent this, so it must always be top of mind. Highlighting the need of retaining velocity can help us with adapting workflows and processes to keep velocity high.

There are a few things that we can do to keep our high velocity. See those described below.

Iterate well

Iterate well on what you are working on. Strive for delivering small merge requests. Write a design docs when you plan to ship multiple iterations that would benefit from additional alignment.

Velocity comes from going fast into the right direction. That is the difference between Speed and Velocity. Direction matters for Velocity, therefore we need to iterate efficiently on the right things at the right time.

For some tips on how to iterate you can check the blog post about 12 shortcuts that are not iteration.

Reduce review time

Be mindful about a code review latency. Do not make authors wait too much before you review their code.

For many people context switching is a very expensive thing to do, which can negatively impact their productivity. Be mindful about that when you are reviewing code or collaborating with team members asynchronously. If a question doesn't need to be answered, or a code suggestion is a minor nitpick which doesn't need to be addressed, then perhaps it is not worth context switching required to do these things.

Reduce process

Reduce the amount of attention to various processes when working on Experimental and Beta features. It is fine to skip certain processes when you are aiming for delivering an Experimental or even Beta feature. Processes can also be automated to reduce their impact on velocity.

For example, if you are working on an Experimental feature, you should aim for reducing the amount of time-consuming processes to the bare minimum. At GitLab we do not ship the process. This might require a pragmatic approach to find the right balance between working on code and doing UX research, customer validation or spending time in long brainstorming sessions. Some of these things will still be needed, but strive for being Efficient and be mindful about the need to keep our velocity high.

For example, if you feel confident that what you're shipping is something that won't result in usability issues, effectively meets a customer need, and/or won't result in workflow issues, then we should have the confidence to move ahead with it without conducting research.

If you can skip a process to gain velocity, consider doing that. We may not be able to apply this rule for GA features, though. Be mindful about the impact of skipping process on customers when you are working on Beta feature. If you are working on an Experimental feature, skipping some processes to gain velocity is recommended after thoughtful consideration.

If you have questions about whether a process is hindering efficiency, is outdated, doesn't scale well, and can be skipped or automated, talk to you manager. When you skip a process, document this decision somewhere (a comment, an issue) and ping people for whom this information might be useful.

Embrace risk

When you are working on Experimental and Beta features it is okay to take some risks. We have many useful tools and automation available that can allow us to take some additional risks to retain high velocity when delivering Experimental features:

1. You can use Feature Flags.
2. You can implement circuit breakers.

3. Use design patterns that increase code isolation.

The more isolated you feature is from the rest of GitLab, the more risks you can take, because it is less likely that a problem in your Experimental feature will affect Generally Available features, or a whole product, in a negative way.

There is one exception for this rule: availability and security. Your code should be always as secure as possible and should not bring GitLab.com down, regardless of the maturity model.

Balance maintenance

Balance the need to refactor code and deliver new features. Use our Efficiency principle as a compass.

Be proactive

If there is a process that has a negative effect on our velocity, be proactive about it: speak up and collaborate with your manager on how can we improve it.

We should always be able to explain why a process exists, and if we're unable to do it, maybe it is "a process for process sake", maybe it is a process of low value, or it does not fit our current scale or our strategic goals anymore. If that is the case perhaps the process could be eliminated, fixed, or automated.

Predictability

Prioritizing technical decisions

Please see the Product Management section that governs how they prioritize work, and also should guide our technical decision making.

Despite the high priority of velocity to our project and our company, there is one set of things we must prioritize over it: GitLab availability & security. Neither we, nor our customers, can run an Enterprise-grade service if we are willing to risk users' productivity and data.

Our hundreds of Engineers collectively make thousands of independent decisions each day that can impact GitLab.com and our users and customers there. They all need to keep availability and security in mind as we endeavor to be the most productive engineering organization in the world. We can only move as fast as GitLab.com is available and secured. Availability of self-managed GitLab instances is also extremely important to our success, and this needs to happen in partnership with our customers' admins (whereas we are the admins for GitLab.com).

For security, we prioritize it more highly by having strict SLAs around priorities labels with security issues. This shows a security first mindset as these issues take precedence in a given timeframe.

Leverage Established Processes

We develop and document processes that are designed to help us in achieving better Results.

For example, the following processes exist to reduce the risk and improve code quality:

- Feature Flags Usage
- Code Review Guidelines

We find opportunities for improvement through analyzing metrics to identify trends, hosting retrospectives (e.g. Group Retrospectives, Iteration Retrospectives), performing Root Cause Analyses, and receiving feedback from team members. Team members are encouraged to identify opportunities to improve our processes and propose solutions, examples of this could be an MR or and issue describing these opportunities.

Everyone can contribute by proposing new processes and improving upon existing processes.

Quality

It is important to remember that quality is everyone's responsibility. Everything you merge to master should be production ready. Familiarize yourself with the definition of done.

Release when it's ready

Our releases page describes our two main release channels:

1. Self-managed users use a monthly self-managed release.
2. GitLab.com uses auto-deploy releases.

As the first of these is a monthly release, it's tempting to try to rush to get something in to a monthly self-managed release. However, this is an anti-pattern. Most issues don't have strict due dates. Those that do are exceptions, and should be treated as such.

Due date pressure logically leads to a few outcomes:

1. People are at increased risk of burnout.
2. We may compromise on our definition of done.

3. We cut scope.
4. We miss the due date.

Only the last two outcomes are acceptable as a general rule. Missing a 'due date' in the form of an assigned milestone is often OK as we put velocity above predictability, and missing the monthly self-managed release does not prevent code from reaching GitLab.com.

For these reasons, and others, we intentionally do not define a specific date for code to be merged in order to reach a self-managed monthly release. The earlier it is merged, the better. This also means that:

1. We don't want merge request authors to work extra hours or otherwise rush to meet a due date.
2. We don't want reviewers and maintainers to be put under pressure to do anything other than meet the regular SLOs.

If it is essential that a merge request make it in a particular release, this must be communicated well in advance to the engineer and any reviewers, to ensure they're able to make that commitment. If a severe bug needs to be fixed with short notice, it is better to revert the change that introduced it than to rush, or even to delay the release until the fix is ready.

In general, there is no need to change any behavior close to the self-managed release.

Dogfooding

We dogfood everything. Based on our product principles, it is the Engineering division's responsibility to dogfood features or do the required discovery work to provide feedback to Product. It is Product's responsibility to prioritize improvements or rebuild functionality in GitLab.

Dogfooding Antipatterns

An easy antipattern to fall into is to resolve your problem outside of what the product offers. Dogfooding is not:

1. Building a bot outside of GitLab.
2. Writing scripts that leverage the GitLab API (if the functionality is on our roadmap and could be shipped within the GitLab Project).
3. Using a component of GitLab that is not part of our components or managed apps.
4. Using templates or repos that are not part of the default UI (having to type or copy-paste to add them).

5. Not deprecating the non-GitLab product or part as you introduce the functionality in GitLab, we need to eat the dogfood, not just serve it.
6. Using it for something that isn't crucial or essential.
7. Delaying until every single feature you use now is in the GitLab product.
8. Not starting it because using a non-GitLab and GitLab product at the same time introduces overhead.
9. Trying to move the most complex part first, instead please iterate and move the simplest first.
10. Building it outside of GitLab because "It is just a prototype", "We're not sure it will work", or "We're not sure it is better than the existing functionality"

Dogfooding Process

Follow the dogfooding process described in the Product Handbook when considering building a tool outside of GitLab.

Code Quality and Standards

We need to maintain code quality and standards. It's very important that you are familiar with the Development Guides in general, and the ones that relates to your group in particular:

- UX Guides
- Backend Guides
- Frontend Guides
- Database Guides

Please remember that the only way to make code flexible is to make it as simple as possible.

Nearby Cats (@BaseCase) wrote on January 16, 2019:

> A lot of programmers make the mistake of thinking the way you make code flexible is by predicting as many future uses as possible, but this paradoxically leads to *less* flexible code. The only way to achieve flexibility is to make things as simple and easy to change as you can.

UI consistency and maintainability across Rails and VueJS Frontend

Part of our engineering culture is to keep shipping so users and customers see significant new value added to GitLab.com or their self-managed instance. To support rapid development, we

choose pragmatically the right technology. As each view is unique, we should equally respect our HAML and Vue codebase and make an educated choice per view as to which framework will enable the most consistency and maintainability.

It's important to keep in mind that when building complex applications, there are many factors to consider such as the fully planned feature to avoid situations where we build an MVC in HAML only to later need to re-write it in Vue due to growing complexity.

To promote visual consistency and an accessible UI, we should always aim to use simple and reusable UI components provided by the GitLab UI component library both in Vue and HAML views. We implement GitLab UI components based on our Pajamas design system and currently these are mostly in Vue, however, we provide adapters that allow us to use a few simple components in HAML as well.

If a GitLab UI component is not available on HAML due to its intrinsic complexity, this is a sign that you should implement your feature using Vue instead.

When to use VueJS

- You are looking to use an already established component from the (gitlab-ui) library.
- You are building a complex feature that will require multiple complex components which are connected to provide interactivity in a single page application.
- You find yourself enhancing a HAML view's behavior with vanilla JavaScript / jQuery (reactive or asynchronous behavior).
- You require medium or larger client-side interactivity.
- You want to build a realtime application.

When to use Rails

- You require a simple, static view.
- You do not need to use multiple components or state management.
- You do not need to use any of the complex components from the (gitlab-ui) library (this does not extend to simple components which can be used in HAML such as labels).

Complex Components

A complex component denotes a type of component that cannot be used easily in our HAML files. This might be due to in-built state management, CSS or dynamic behaviour that rapidly becomes a maintainability burden inside HAML. An example of such a component would be our Table component.

- backend
- frontend
- use GraphQL everywhere
- the importance of velocity
- engineering proposed initiative

Last modified October 14, 2026: PS: retire the professional-services-engineering workflows/ stubs (`0c3f0ab3`)

View page source - Edit this page - please contribute. (cc) BY-SA